

Keystone Connect Production Technical Blueprint (First-Person)

1) My Objective

I am building Keystone Connect as a production-grade contractor discovery platform that is fast, stable, secure, and usable across desktop and mobile. My goal is to give users a consistent way to search for contractors by location and construction category, review results, suppress irrelevant businesses, save preferred companies, and export clean project-ready data.

I want the app to feel simple on the surface, but underneath it needs to be reliable, observable, and ready to scale without turning into a fragile pile of one-off fixes.

2) My Current Baseline

Right now, Keystone Connect is a Node.js application that serves both the UI and API from a single process.

My current baseline is:

- Runtime: Node `24.x`
- Server: `/Users/mcdowell/Desktop/temp files/Keystone Connect/src/web-server.js`
- Search engine: `/Users/mcdowell/Desktop/temp files/Keystone Connect/src/search-engine.js`
- Taxonomy: `/Users/mcdowell/Desktop/temp files/Keystone Connect/data/taxonomy.json`
- Primary UI: `/Users/mcdowell/Desktop/temp files/Keystone Connect/ui-v2/app.html`
- Current persistence:
 - `/Users/mcdowell/Desktop/temp files/Keystone Connect/data/irrelevant-filters.json`
 - `/Users/mcdowell/Desktop/temp files/Keystone Connect/data/preferred-results.json`

My current app routes are:

- App:
 - `/`
 - `/v2`
 - `/ui-v2`
 - `/ui-v2/app.html`
 - fallback `/v1`
- APIs:

- `POST /api/search`
- `GET /api/query-categories`
- `GET /api/location-suggest`
- `GET /api/location-city`
- `GET /api/reverse-zip`
- `GET /api/preferred`
- `POST /api/preferred`
- `POST /api/irrelevant`
- `POST /api/csv` (legacy)
- `GET /api/ping`

3) How I Expect the Product to Work

3.1 Search flow

When a user searches, I expect the system to do the following:

1. Accept a location from typed input or approximate current location.
2. Accept a taxonomy selection, preferred results view, or manual override.
3. Accept a radius and search mode.
4. Resolve the user's location into a valid search center.
5. Build the correct search jobs based on the selected engine mode.
6. Execute those jobs against Google Places with proper retry logic.
7. Deduplicate, filter, rank, and clamp results to the requested radius.
8. Return a clean result set that matches the UI sort and export behavior.

3.2 Result controls

I want result management to be intentional and persistent:

- Preferred results should be saved and reusable.
- Irrelevant removals should apply to the specific query context they belong to.
- Prior result filtering should help users expand a search radius without re-reviewing businesses they already saw.

3.3 Export behavior

I want exports to be useful immediately, not cleanup projects after the fact. My export should:

- reflect the visible result set,
- honor active filters,
- preserve the selected category context,
- and generate a clean `.xlsx` file name based on location and category.

4) My Search Architecture

I currently support multiple search modes because one mode alone does not balance precision and coverage well enough.

4.1 API mode

This is my tighter, more controlled mode. I use taxonomy-driven query generation, weighted child profiles, and stricter filtering so results stay closer to the intended business class.

4.2 APIB mode

This is my middle-ground mode. I use it when I want broader coverage than strict API mode without going as loose as GGL. It gives me a better balance when a business exists but is weakly categorized.

4.3 GGL mode

This is my broadest mode. I use it when I need to cast a wider net and accept noisier data in exchange for better recall.

5) How I Rank and Filter Results

I do not treat Google results as truth. I treat them as candidate records that need ranking and cleanup.

My current ranking model considers:

- expected type matches,
- category hint matches,
- business name relevance,
- website presence,
- phone presence,
- rating count,
- rating quality,
- proximity,
- and child/category weighting.

I also apply:

- hard excludes for obviously wrong result classes,
- soft penalties for low-confidence matches,
- radius enforcement,
- and query-signature suppression logic.

My intent is to return the best contractor candidates first without flooding the user with junk links, residential mismatches, or irrelevant service businesses.

6) My Production Target Architecture

I want the production version to stay operationally simple but structurally correct.

My target service model is:

1. A web/API service for search and UI delivery
2. A real datastore for mutable state
3. An optional async worker for slow enrichments like email discovery
4. Logging, metrics, and alerting around the whole system

I do not want production behavior to depend on runtime-edited JSON files forever. That is acceptable for prototyping, but not for a serious internal system.

7) My Data Model Direction

I want to replace file-based persistence with relational storage.

At minimum, I want these server-side entities:

- `projects`
- `search\_runs`
- `search\_results`
- `suppressed\_results`
- `preferred\_results`
- `audit\_events`

My preferred production database is PostgreSQL.

That gives me:

- proper concurrency,
- search history,
- project scoping,
- user accountability,
- and a path to shared team workflows without spreadsheet collisions.

8) My Security Model

I want Keystone Connect to be simple for users, but strict about secrets and mutation paths.

8.1 Secrets

I will keep `GOOGLE\_MAPS\_API\_KEY` in environment configuration only. I do not want it hardcoded into the app, checked into Git, or exposed through export flows.

8.2 API protection

I want:

- request validation,
- bounded payloads,
- engine mode validation,
- rate limits on search,
- and clear server-side rejection of malformed or abusive requests.

8.3 Retention

Even though this is business contact data, I still want retention rules for:

- search history,
- suppression actions,
- preferred result changes,
- and future team activity logs.

9) My Reliability Standard

I want this app to feel dependable, not experimental.

My production reliability goals are:

- API availability of `99.9%`

- predictable search behavior across modes
- partial-result handling when upstream calls fail
- visible health checks
- useful error messages instead of vague breakage

I also want the app to degrade gracefully:

- if Google rate-limits me,
- if one query branch fails,
- or if email enrichment is unavailable.

10) My Frontend Standard

I want the UI to feel polished, consistent, and stable across mobile and desktop.

That means:

- consistent spacing,
- predictable field behavior,
- no random browser-specific control drift,
- stable FAQ behavior,
- clean category menus,
- consistent control alignment,
- and result rows that are readable at a glance.

I also want the frontend to be split more cleanly over time:

- state,
- API client,
- rendering,
- interactions,
- and theme tokens

Instead of letting all behavior live forever inside one HTML file.

11) My Quality Engineering Plan

I do not want production confidence to depend on manual memory.

I want automated coverage for:

- taxonomy parsing,
- query building,
- search signatures,
- ranking,
- suppression logic,
- preferred result behavior,
- and `.xlsx` export structure.

I also want a regression pack of known business checks by city/category so I can detect when a valid company disappears from a result class that used to find it.

12) My Release Process

I want releases to be disciplined.

My release path should be:

1. Validate code and dependencies
2. Run tests
3. Build artifacts
4. Smoke-test the app
5. Deploy
6. Verify health endpoints
7. Keep rollback clear and simple

For desktop builds, I also want packaging cleanup to be part of the process so only the current sendable files remain visible to nontechnical users.

13) My Desktop Packaging Standard

I want the desktop version to feel like a real internal product, not a developer handoff.

That means:

- clean `Run` and `Share` folders,
- compatibility builds separated,
- friendly file names,

- predictable launcher behavior,
- and minimal user confusion about what to install.

My desktop packaging target is:

- Mac (M-chip) current build
- Windows (Most PCs) current build
- compatibility builds separated for edge cases

My desktop launcher behavior is:

- start on `127.0.0.1`
- prefer port `8788`
- move to the next open port automatically if needed
- open the system browser only after the local server is healthy

I also want the packaging workflow to automatically archive older artifacts and keep only the latest mainstream files exposed.

14) My Team Workflow Direction

I know users will eventually want shared, living contractor workbooks by division, project, and calling status.

I do not want that to depend on multiple people editing the same raw spreadsheet at once without system coordination. My long-term approach is:

- keep canonical data in the application,
- track statuses server-side,
- generate exports as views,
- and use spreadsheets as deliverables, not as the database.

That gives me a cleaner future path for:

- project-specific suppression,
- no-duplicate expansion searching,
- contact workflow status,
- and master workbook generation by CSI division.

15) My Roadmap

Phase 1: Stabilize

I lock down the current app shape:

- request validation
- structured error handling
- smoke checks
- predictable package outputs

Phase 2: Persist

I replace runtime JSON persistence with Postgres:

- preferred results
- suppressed results
- search memory
- project scoping

Phase 3: Scale

I reduce API waste and latency:

- caching
- async enrichment
- better rate limit handling
- search replay reduction

Phase 4: Operationalize

I make it team-ready:

- roles
- audit history
- project workspaces
- governed exports
- admin controls

16) My Definition of Production-Grade v1

I will consider Keystone Connect production-grade when:

- mutable runtime data is no longer stored only in JSON files,

- automated tests protect core search and export behavior,
- logs and health checks are in place,
- request validation and rate limiting are active,
- secrets are properly managed,
- deployment and rollback are documented,
- and team workflows can scale without manual spreadsheet chaos.

17) Bottom Line

Keystone Connect is not just a search UI to me. I am building it as a controlled contractor intelligence system: one that starts simple for users, but is engineered underneath to support repeatable searching, cleaner decision-making, team workflows, and future project-level data management without breaking trust or usability.